PROJECT REPORT

PRESENTATION

Sina Abbaszadeh

40030212094

In the Name of God

Computer Vision Course Project
Project Title: Iranian National ID Card Scanning and Information Extraction into Excel
Group Member:
Sina Abbaszadeh
Course Instructor:
Professor Morteza Ghazali

Explanation of camera.html

The file camera.html contains the frontend interface of the project. In this .section, its structure and main components are explained

Lines 1 to 4 define the essential HTML structure required for a standard web .page

.Line 5 defines the mobile viewport settings

width=device-width sets the page width equal to the width of the user's device, which helps make the camera interface responsive .on mobile phones

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta name="viewport" content="width=device-width, initial-s
    <title>کارت ملی</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        html,
        body {
            width: 100%;
            height: 100%;
            overflow: hidden;
            position: fixed;
        }

        #root {
            width: 100%;
            height: 100%;
        }
    </style>
    <script crossorigin src="https://unpkg.com/react@18/umd/reac
    <script crossorigin src="https://unpkg.com/react-dom@18/umd/
    <script src="https://unpkg.com/@babel/standalone/babel.min.j
    <script src="https://cdn.tailwindcss.com"></script>
</head>
```

initial-scale=1.0: This property defines the initial zoom level of the page when it is first loaded by the browser. Setting this value to 1.0 ensures that the page is displayed at its normal scale, without being zoomed in or out. Therefore, the visual width of the page remains consistent with the device width defined by width=device-width.

maximum-scale=1.0, user-scalable=no: These settings prevent the user from manually zooming in or out. This is especially useful in a camera-based interface, where a fixed and stable layout is important for accurate card positioning and image capture.

Lines 8 to 25 define the main visual styling of the page. For example, overflow: hidden hides any content that extends beyond the defined boundaries of an element. This prevents unwanted scrolling or visual overflow. The property position: fixed keeps the selected element fixed in place on the screen, regardless of page movement.

Line 11:
This line sets the box-sizing behavior so that padding and border are included in the total width and height of each element.

Lines 27 to 30:
These lines load the main external dependencies of the frontend application. React and ReactDOM provide the core functionality for building the user interface. Babel enables the browser to process modern JavaScript and JSX syntax. Tailwind CSS is used for styling and layout design.

In the following section, the HTML structure is explained step by step. When an element calls a JavaScript function, that function is also explained in the relevant part.

```
200             return (
201                 <div className="relative w-full bg-black overflow-hidden" s
202                     {/* camera */}
203                     <video
204                         ref={videoRef}
205                         autoPlay
206                         playsInline
207                         muted
208                         onLoadedMetadata={handleMetadataLoaded}
209                         className="absolute inset-0 w-full h-full object-co
210                         style={{ display: capturedImage ? 'none' : 'block'
211                     />
212
213                     {capturedImage && (
214                         <div className="absolute inset-0">
215                             <img
216                                 src={capturedImage}
217                                 alt="عکس گرفته شده"
```

The main div is first assigned a black background. Its width is set to cover the full page, and its height is set to 100dvh. This means that the div occupies the full visible height of the viewport, independent of the parent element's content.

Line 203:
This line starts the video element, which is used to display the live camera stream.

Line 204:
This line connects the video element to a JavaScript reference named **videoRef**. This allows us to access the actual properties of the video element, such as videoWidth and videoHeight, inside functions like capturePhoto.

Line 205:
autoplay: This tells the video to start playing automatically as soon as it is loaded.

Line 206:
playsInline: In mobile browsers, this ensures that the video does not open in full-screen mode and instead plays directly inside the web page.

Line 207:
muted: This mutes the video.

Line 208:
When the video metadata, such as its dimensions, has been loaded, the handleMetadataLoaded function is called.

Line 209:
These lines define the visual styling of the video element. For example, object-cover makes the camera feed fill the entire available space regardless of its original aspect ratio. In other words, the video element is forced to match the shape of the page frame, even if parts of the video edges need to be cropped.

Line 210:
This is a conditional rendering line. It displays the video element only if the capturedImage variable has not been set yet, meaning that no photo has been captured.

Lines 213 to 220:
If capturedImage has a value, meaning that a photo has already been taken, an img element is displayed using capturedImage as its image source.

```
42        const [isVideoReady, setIsVideoReady] = useState(false); //Camera readiness status
```

```
    const handleMetadataLoaded = () => {
        setIsVideoReady(true);
    };
```

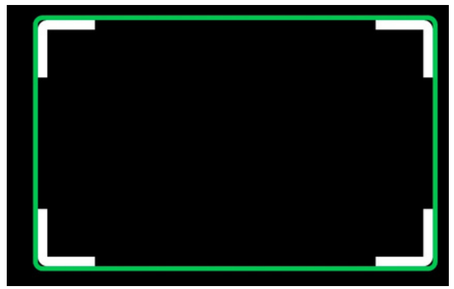
The function called on line 208 is implemented by setting the value of the isVideoReady variable to true. Initially, this variable is set to false.

The value of this variable will be used later in the program to determine whether the video stream is ready and whether certain actions, such as taking a photo, should be enabled.

```jsx
{!capturedImage && (
    <>
        {/* help box */}
        <div className="absolute inset-0 flex items-center justify-center pointer-events-none">
            <div ref={frameRef} className="relative" style={{ width: FRAME_WIDTH, height: FRAME_HEIGHT }}>
                {/* main box with border */}
                <div className="absolute inset-0 border-4 border-green-500 rounded-lg shadow-2xl z-20">
                    <div className="absolute top-0 left-0 w-12 h-12 border-t-8 border-l-8 border-white rounded-tl-lg"></div>
                    <div className="absolute top-0 right-0 w-12 h-12 border-t-8 border-r-8 border-white rounded-tr-lg"></div>
                    <div className="absolute bottom-0 left-0 w-12 h-12 border-b-8 border-l-8 border-white rounded-bl-lg"></div>
                    <div className="absolute bottom-0 right-0 w-12 h-12 border-b-8 border-r-8 border-white rounded-br-lg"></div>
                </div>
            </div>

            {/* box text */}
            <div className="absolute bottom-32 left-1/2 transform -translate-x-1/2 text-white text-lg bg-black bg-opacity-70 px-6 py-3 rounded-full z-30" style={{ direction: 'rtl' }}>
                کارت ملی را داخل کادر قرار دهید
            </div>
        </>
)}
```

This section is executed only when no photo has been captured yet.
If no photo has been taken, a full-screen rectangle is placed over the video element using the classes absolute and inset-0. Inside this overlay, another rectangle is created with the predefined dimensions FRAME_WIDTH and FRAME_HEIGHT.
Inside this frame, a second rectangle with a green border is displayed. Four additional small rectangles are placed at the corners of this green frame, each with white borders. These corner markers help guide the user to position the ID card correctly inside the frame.



Outside these rectangles, there is another rectangular box that contains the instruction: “Place the national ID card inside the frame.”
This message guides the user to position the card correctly before capturing the image.


```jsx
{/* massage error */}
{error && (
    <div className={`absolute top-4 left-1/2 transform -translate-x-1/2 w-11/12 max-w-md ${error.startsWith('✅') ? 'bg-green-500' : 'bg-red-500'} text-white p-4 rounded-lg text-center z-40`} style={{ direction: 'rtl' }}>
        {error}
    </div>
)}

<canvas ref={canvasRef} className="hidden" />
```

In this section, if an error exists, the error message is displayed inside a rectangular box. The value of the error message is set inside the related functions when an error occurs.

error.startsWith: This condition checks whether the error message starts with a specific prefix. If it does, the background color is set to green; otherwise, it is set to red.

Canvas Element:

The video element displays the live camera feed. Since this is a live stream, we cannot directly capture a still image from it in the same way we use a normal image file. The canvas element acts like a hidden drawing board. When we want to capture a photo from the camera feed, the current frame of the video is first drawn onto this hidden canvas. After the frame is drawn on the canvas, JavaScript can convert it into an image file format such as JPEG or PNG and store it in the capturedImage variable.

A reference named canvasRef is assigned to the canvas element so that we can access and use this element later in the program when needed.

```
{/* bottons */}
<div className="absolute bottom-6 left-1/2 transform -translate-x-1/2 w-11/12 max-w-md z-30">
    {!capturedImage ? (
        <div className="flex gap-4 justify-center items-center">
            {/* Botton to capture photo  */}
            <button
                onClick={capturePhoto}
                className={`p-6 rounded-full transition-all shadow-2xl pointer-events-auto ${isVideoReady ? 'bg-white hover:bg-gray-200' : 'bg-gray-400 cursor-not-allowed'}`}
                disabled={!isVideoReady}
            >
                <div className="w-16 h-16 bg-red-500 rounded-full flex items-center justify-center">
                    <img src="photo/takePhoto.png" alt="گرفتن عکس" className="w-8 h-8 " />
                </div>
            </button>
        </div>
    ) : (
```

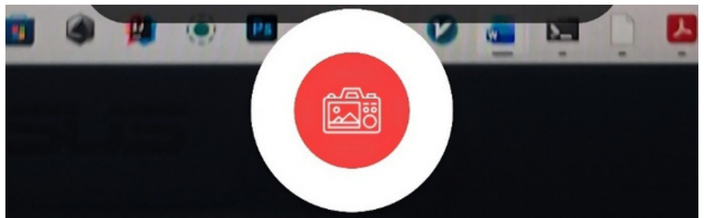
This section contains the buttons used in the camera interface.

If no photo has been captured yet, a button is displayed on top of the video field. When the user clicks this button, the capturePhoto function is executed. This function will be explained in the following section.

disabled={!isVideoReady}: This means that if the camera is not ready yet, the button will be disabled.

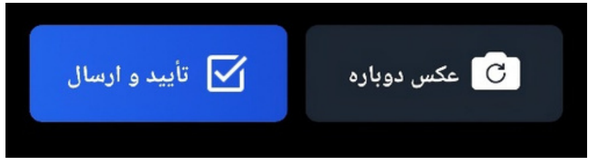
Also, if the value of isVideoReady is false, the mouse cursor will appear as a “not allowed” cursor when the user hovers over the button. This visually indicates that the button cannot be used until the camera is ready.

Lines 265 to 269: **These lines place the camera icon inside the capture button.**



```
270                     ) : (
271                         <div className="flex gap-3 pointer-events-auto">
272                             {/* botton to retake photo */}
273                             <button
274                                 onClick={retakePhoto}
275                                 className="flex-1 bg-gray-800 bg-opacity-90 hover:bg-gray-700 text-white font-bold py-4 px-6 rounded-lg transition-all flex items-center justify-center gap-2"
276                             >
277                                 <img src="photo/retakePhoto.png" alt="عکس دوباره" className="w-8 h-8 " />
278                                 <span style={{ direction: 'rtl' }}>عکس دوباره</span>
279                             </button>
280
281                             {/* botton to upload photo */}
282                             <button
283                                 onClick={uploadPhoto}
284                                 className="flex-1 bg-gradient-to-r from-blue-500 to-blue-600 hover:from-blue-600 hover:to-blue-700 text-white font-bold py-4 px-6 rounded-lg transition-all flex items-center justify-center gap-2"
285                             >
286                                 <img src="photo/tickBox.png" alt="تایید و ارسال" className="w-8 h-8 " />
287
288                                 <span style={{ direction: 'rtl' }}>تأیید و ارسال</span>
289                             </button>
290                         </div>
291                     )}
292                 </div>
```

in this section, if a photo has already been captured, two buttons are displayed.
The first button is the **“Retake Photo”** button. When the user clicks this button, the retakePhoto function is executed.
The second button is the **“Confirm and Send”** button. When the user clicks this button, the uploadPhoto function is executed.



Before explaining these functions, there is one important point that should be reviewed first

Lines 52 to 54:
In these lines, the references are defined. At this point, they do not point to any specific elements yet.

Lines 83 to 85:
In these lines, useEffect is used. The dependency array is set to [], which means that the content inside this useEffect will run only once, when the component is first created or when the program starts.
In this case, the function called inside useEffect is startCamera, so the camera starts automatically when the component is mounted.

startCamera Function:
The startCamera function is responsible for requesting camera access from the user and then displaying the live camera stream inside the video element of the application.

Line 58:
This line clears the value of the Error variable. Error is a variable defined in the program to store and display error messages when something goes wrong.

```
52          const videoRef = useRef(null);
53          const canvasRef = useRef(null);
54          const frameRef = useRef(null);
55
56          const startCamera = async () => {
57              try {
58                  setError('');
59
60                  if (!stream) {
61                      const mediaStream = await navigator.mediaDevices.getUserMedia({
62                          video: {
63                              facingMode: 'environment',
64                              width: { ideal: 4096 },
65                              height: { ideal: 2160 }
66                          },
67                          audio: false
68                      });
69
70                      setStream(mediaStream);
71                      if (videoRef.current) {
72                          videoRef.current.srcObject = mediaStream;
73                      }
74                  } else if (videoRef.current && !videoRef.current.srcObject) {
75                      videoRef.current.srcObject = stream;
76                  }
77              } catch (err) {
78                  setError('دسترسی به دوربین امکان‌پذیر نیست. لطفاً مجوز دوربین را بدهید.');
79                  console.error('خطا در دسترسی به دوربین:', err);
80              }
81          };
82
83          useEffect(() => {
84              startCamera();
85          }, []);
```

The startCamera function handles two main situations: the first case is obtaining a new camera stream, and the second case is reusing an existing stream.

Line 60:

This line checks whether the stream state variable is empty. If it is empty, it means that the camera is being started for the first time.

Line 61:

This line uses the browser's standard API for requesting access to media devices, such as the camera and microphone.

Line 63:

This line requests access to the rear camera, which is usually the main camera of the device.

Lines 64 and 65:

These lines attempt to request the highest possible video resolution, such as 4K, in order to improve the quality of the captured images.

Line 66:

This line specifies that microphone access is not required.

Line 70:

The received video stream is stored in the stream state variable.

Lines 71 to 73:

The received mediaStream is connected to the video element through videoRef.current.srcObject so that the live camera feed can be displayed on the screen.

Lines 74 to 76:

If the stream has already been created and exists, it is reconnected to the video element. This makes the startup process faster because the program does not need to request camera permission from the user again.

Now, we will examine the functions.

capturePhoto Function:

As mentioned earlier, this function is executed when the user clicks the camera button.

```
91          const capturePhoto = () => {
92              const video = videoRef.current;
93              const canvas = canvasRef.current;
94              const frame = frameRef.current;
95
96              if (!video || !canvas || !frame) {
97                  setError('خطا: المان‌های دوربین کامل لود نشده اند.');
98                  return;
99              }
100
101             const videoWidth = video.videoWidth;
102             const videoHeight = video.videoHeight;
103
104             if (videoWidth === 0 || videoHeight === 0 || !isVideoReady) {
105                 setError('دوربین هنوز آماده نیست. لطفا چند لحظه صبر کنید.');
106                 return;
107             }
```

Lines 91 to 94:

These lines extract the references from videoRef, canvasRef, and frameRef.

Lines 101 and 102:

These lines obtain the actual resolution of the video received from the camera, for example 1920 × 1080. These dimensions are often different from what the user sees on the screen.

```
109         const containerWidth = video.clientWidth;
110         const containerHeight = video.clientHeight;
111
112         const videoAspectRatio = videoWidth / videoHeight;
113         const containerAspectRatio = containerWidth / containerHeight;
114
115         let displayedWidth, displayedHeight, offsetX, offsetY;
116
117         if (videoAspectRatio > containerAspectRatio) {
118             displayedHeight = containerHeight;
119             displayedWidth = containerHeight * videoAspectRatio;
120             offsetX = (containerWidth - displayedWidth) / 2;
121             offsetY = 0;
122         } else {
123             displayedWidth = containerWidth;
124             displayedHeight = containerWidth / videoAspectRatio;
125             offsetX = 0;
126             offsetY = (containerHeight - displayedHeight) / 2;
127         }
```

Lines 109 and 110:

These lines store the displayed dimensions of the video element on the user's screen.

Lines 112 and 113:

These lines calculate the actual aspect ratio of the video and the aspect ratio of the displayed video element.

Line 115:

Four variables are defined here.

At this point, we want to determine an important value, but before that, it is useful to review one principle.

As explained earlier, the video element uses CSS to cover the entire screen, for example with object-cover. Because of this, the actual dimensions of the video, which come from the camera resolution, may not match the dimensions displayed on the screen.

This block of code uses JavaScript to calculate how much the video has been scaled and how much of its edges have been cropped.

Line 117:

In this case, the video is wider than the display frame.

If the aspect ratio of the video is greater than the aspect ratio of the display frame, for example when a 16:9 video is displayed inside a narrower 4:3 frame, the video must be scaled based on the height of the frame so that it fully covers the frame vertically.

Line 118:

The displayed height of the video is set equal to the height of the frame.

Line 119:

Since the height has been adjusted, the new width must be calculated while preserving the original aspect ratio of the video.

Because the original aspect ratio is preserved, the calculated width becomes larger than the width of the display frame. For example, suppose the scaled video width is 533 pixels while the frame width is 400 pixels. In this case, the scaled video extends beyond the frame by a total of 133 pixels horizontally.

This overflow is automatically cropped by the CSS property object-cover, so only the visible 400-pixel-wide part is displayed.

Now, we need to calculate the amount of this overflow. To do this, we subtract the scaled video width from the frame width. The resulting value represents the total number of pixels cropped from the left and right sides. We then divide this value by 2 to determine how many pixels are cropped from each side.

```
129             const frameRect = frame.getBoundingClientRect();
130
131             const scaleX = videoWidth / displayedWidth;
132             const scaleY = videoHeight / displayedHeight;
133
134             const relativeX = frameRect.left - offsetX;
135             const relativeY = frameRect.top - offsetY;
136
137             const cropX = relativeX * scaleX;
138             const cropY = relativeY * scaleY;
139
140             const cropWidth = frameRect.width * scaleX;
141             const cropHeight = frameRect.height * scaleY;
142
143             canvas.width = cropWidth;
144             canvas.height = cropHeight;
145
146             const context = canvas.getContext('2d');
147             context.drawImage(
148                 video,
149                 cropX, cropY, cropWidth, cropHeight,
150                 0, 0, cropWidth, cropHeight
151             );
```

Line 122:

In this case, the display frame is wider than the video.

If the aspect ratio of the video is smaller than or equal to the aspect ratio of the display frame, for example when a 9:16 video is displayed inside a wider 16:9 frame, the video must be scaled based on the width of the frame so that it fully covers the frame horizontally.

Line 124:

As a result, the displayed height of the video will logically become larger than the height of the frame. Therefore, the video will be cropped from the top and bottom.

Line 129:

The getBoundingClientRect() method provides precise geometric information about the frame element, which is the green guide frame for the national ID card. This information includes the position and dimensions of the element relative to the browser viewport. As a result, frameRect contains values such as left, top, width, and height in CSS pixels. The method returns a DOMRect object that describes the element's size and position relative to the viewport. :contentReference[oaicite:0]{index=0}

Lines 131 and 132:

These lines calculate the scale ratio between the real camera coordinate system and the coordinate system displayed on the screen.

Lines 134 and 135:

These lines calculate the position of the top-left corner of the guide frame relative to the top-left corner of the scaled video, which has already been shifted using offsetX and offsetY.

In other words, suppose frame.left is equal to 100. This means that the green frame is 100 pixels away from the left side of the displayed frame. However, we need to determine its distance from the actual scaled video.

Previously, we calculated how much of the scaled video was cropped from the left, right, top, and bottom. For the left and right sides, this value was stored in offsetX. Since offsetX can be negative, subtracting it from frame.left actually adds its absolute value. By doing this calculation, we obtain the distance of the left side of the green frame from the left side of the scaled video.

To crop and save a specific region of the image, we need to define the source region. This is done by specifying the starting point, which is represented by the X and Y coordinates, and the crop dimensions, which are the width and height. This allows the Canvas drawing engine to extract exactly the desired part of the image.

The starting points are cropX and cropY. The height of the cropped region is cropHeight, and the width of the cropped region is cropWidth.

cropX and cropY are calculated as follows:
We have already obtained the distance of the guide frame from the starting point of the scaled video. However, this distance, such as relativeX, is still measured in screen pixels, or CSS pixels.
Therefore, we need to multiply this distance by the scale factor, such as scaleX, in order to convert it into the real camera pixel coordinate system.

Lines 140 and 141:

These lines determine the final dimensions of the cropped image in real camera pixels. In other words, this operation converts the smaller displayed size on the screen into the larger, higher-quality resolution of the actual camera image.

Lines 143 and 144:

The dimensions of the hidden canvas are set exactly equal to the final dimensions of the cropped image. This ensures that the canvas has only the required amount of space to store the cropped image.

Line 146:

The context object is obtained. This object is the main tool used for drawing and manipulating pixels on the canvas.

Line 148:

The camera video element is used as the image source.

Line 149:

The values cropX, cropY, cropWidth, and cropHeight tell the drawImage engine where to start cropping from and what size of region should be taken from the high-resolution camera image.

Line 150:

The values 0, 0, cropWidth, and cropHeight tell the drawing engine to place the cropped image starting from the top-left corner of the canvas and to fill the entire canvas.
At this point, the canvas contains a high-quality image that has been cropped exactly from inside the user’s guide frame.

```
    const imageData = canvas.toDataURL('image/png');
    setCapturedImage(imageData);
    setError('');
};
```

Line 153:

This line reads the pixel data from the canvas and converts it into a Data URL string. In this format, the image data is encoded as Base64 text.
The resulting Data URL can be stored in a variable and used directly in HTML, for example as the source of an image element.

Line 154:

This line stores the image data string in the capturedImage state variable.
This causes the component to re-render. As a result, the conditional rendering logic is activated: the camera shutter button is removed, and the “Confirm” and “Retake Photo” buttons are displayed instead.
The reason this part of the code is relatively complex is that the real camera resolution does not match the size displayed on the screen using CSS. The goal is to coordinate these two different coordinate systems: the actual camera pixel space and the displayed screen space.

Now, we will examine the retakePhoto function.

As explained earlier, this function is called when a photo has already been taken and the user clicks the “Retake Photo” button.
This function sets the value of the capturedImage variable to null. As a result, the elements whose display condition depends on capturedImage being empty will be shown again.
Executing this function removes the captured image preview and returns the application to the live camera mode.

```
159         const retakePhoto = () => {
160             setCapturedImage(null);
161             setError('');
162         };
```

```
164         const uploadPhoto = async () => {
165             if (capturedImage) {
166                 setError('در حال ارسال عکس...');
167 
168                 const response = await fetch(capturedImage);
169                 const blob = await response.blob();
170 
171                 const formData = new FormData();
172                 formData.append('imageFile', blob, `cropped_id_card_${Date.now()}.png`);
173 
174                 try {
175                     // request to server
176                     const uploadResponse = await fetch(SERVER_URL, {
177                         method: 'POST',
178                         body: formData
179                     });
180 
181                     if (uploadResponse.ok) {
182                         setError('✅ عکس با موفقیت به سرور ارسال و ذخیره شد.');
183                     } else {
184                         const errorText = await uploadResponse.text();
185                         setError(`خطا در ارسال: ${uploadResponse.status} - ${errorText.substring(0, 50)}...`);
186                     }
187 
188                 } catch (err) {
189                     setError('خطای اتصال به سرور. مطمئن شوید سرور در حال اجراست.');
190                     console.error('خطای ارسال:', err);
191                 }
192             }
193         };
```

Finally, we examine the uploadPhoto function. This function is executed when the user clicks the “Confirm and Send” button:

If a captured image exists, the function begins the upload process. The image extracted from the canvas is stored in the capturedImage variable as a long text string in Data URL format. This string usually starts with:
...
This means that the image is encoded as Base64 text and represented as a PNG image inside the Data URL.

Finally, we will examine the uploadPhoto function, which is called when the user clicks the “Confirm and Send” button.
In this function, if a captured photo exists, the following process is performed:
The image extracted from the canvas and stored in the capturedImage variable is a very long text string that starts with:
...
This format is called a Data URL. Browsers do not treat this string as a real file; instead, they treat it simply as text.
In order to send this image to the server like a real file, for example as a PNG file, we need to convert it into a standard web binary format called a Blob.

Line 168:
When fetch is executed on a Data URL, the browser treats that text string as a data source and reads it.

Line 169:
The .blob() method extracts the binary data from the Response object and converts it into a Blob object.

Line 171:
FormData is a standard web object used to create data in multipart/form-data format. This format is commonly used for uploading files through HTML forms, and servers can easily process it.

Line 172:
formData.append(...): This adds the file to the data package that will be sent to the server.
'**imageFile':** This is the field name expected by the server. In other words, it is the key that the server uses to find the uploaded file.
blob: This is the file data, meaning the converted image.
...png: This is the filename sent to the server. It includes a timestamp generated by Date.now() to make the filename unique.
Finally, lines 176 to 178 send this data to the specified server address.

```python
import os
import re
from datetime import datetime

import cv2
import numpy as np
import pandas as pd

from fastapi import FastAPI, UploadFile, File, HTTPException, Request
from fastapi.responses import FileResponse, JSONResponse
from fastapi.middleware.cors import CORSMiddleware

import easyocr
import uvicorn


try:
    from rapidfuzz import fuzz
except ImportError:
    fuzz = None
```

Now, we will examine the server.py file.

1) Imports and Dependencies

cv2 and NumPy:

These libraries are used for image processing tasks, such as edge detection, cropping, perspective transformation, thresholding, and other image manipulation operations.

pandas:

This library is used to store the extracted results inside the results.xlsx Excel file.

FastAPI:

FastAPI is used to create the backend API and receive the uploaded image file from the frontend.

EasyOCR:

EasyOCR is used for Persian and English optical character recognition.

RapidFuzz optional:

RapidFuzz is used for fuzzy matching of field labels. For example, it can help detect a label such as “family name” even if it is partially or incorrectly recognized by OCR.

If RapidFuzz is not installed, the value of fuzz becomes None. In that case, the fuzzy matching part becomes less effective and the program mainly relies on exact matching or simple text inclusion.

```python
app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

2) CREATING THE APP AND CONFIGURING CORS

In this section, the FastAPI application is created and CORS is configured.

CORS allows frontends from different domains or origins to send requests to this API. This is useful for camera.html and for using the application through a mobile browser, because the frontend and backend may not always be served from exactly the same origin.

3) PATHS AND OUTPUT FILES

```python
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

EXCEL_PATH = os.path.join(BASE_DIR, "results.xlsx")
LAST_RECEIVED = os.path.join(BASE_DIR, "last_received.png")
LAST_WARPED   = os.path.join(BASE_DIR, "last_warped.png")
LAST_DEBUG    = os.path.join(BASE_DIR, "last_debug.png")

# سایز ثابت کارت بعد از warp
WARP_W, WARP_H = 1200, 760
```

results.xlsx:
This file stores the history of extracted records.
last_received.png:
This file stores the last raw image received from the frontend.
last_warped.png:
This file stores the last perspective-corrected image of the card.
last_debug.png:
This file stores the debug image, including detected boxes and regions of interest (ROIs).
WARP_W / WARP_H:
These values define the fixed dimensions of the card after the warping process. Using fixed dimensions is important because it keeps the ROIs stable and consistent in every processed image.

4) OCR READER

.The OCR reader is configured for bilingual Persian and English text recognition, without using GPU acceleration

```python
# OCR
reader = easyocr.Reader(["fa", "en"], gpu=False)
```

LINES 54 TO 95

5) SERVING CAMERA.HTML

The two routes / and /camera.html return the same HTML file that is located next to the script. If the file does not exist, the server returns a 404 error.

6) DEBUG AND DOWNLOAD ENDPOINTS

/last-image:
Displays the last image received by the server.
/last-warped:
Displays the last warped image after perspective correction.
/last-debug:
Displays the last debug image, including boxes and recognized text regions.
/download-excel:
Downloads the Excel file containing the extracted results.
These endpoints are very useful for testing the system and adjusting the ROIs and OCR quality.

7) TEXT AND DIGIT NORMALIZATION PERSIAN/ARABIC TO ENGLISH

Iranian ID cards may contain Persian digits such as ۰۱۲... or Arabic digits such as ٠١٢.... This section converts them into standard Latin digits.
normalize_digits(s):
Converts all digits into Latin digits from 0 to 9.
clean_text(s):
Removes extra spaces and line breaks from the text.

```python
DATE_RE = re.compile(r"(1[3-4]\d{2})\s*[/\-\.\s]\s*(\d{1,2})\s*[/\-\.\s]\s*(\d{1,2})")

AR_LETTERS_RE = re.compile(r"[\u0600-\u06FF]")
def looks_like_persian_name(s: str) -> bool:
    s = clean_text(s)
    if not s:
        return False
    if re.search(r"\d", normalize_digits(s)):
        return False
    # حداقل 2 حرف فارسی/عربی داشته باشد
    return len(AR_LETTERS_RE.findall(s)) >= 2
```

8) BIRTH DATE EXTRACTION

This section detects birth dates with years starting with 13xx or 14xx. It can recognize dates written in formats such as YYYY/MM/DD, or with separators such as hyphens, spaces, and similar variations.

```python
def extract_birth_date(text: str) -> str:
    t = normalize_digits(text)
    m = DATE_RE.search(t)
    if not m:
        return ""
    y = int(m.group(1))
    mo = int(m.group(2))
    d = int(m.group(3))

    # اعتبارسنجی منطقی
    if not (1300 <= y <= 1499):
        return ""
    if not (1 <= mo <= 12):
        return ""
    if not (1 <= d <= 31):
        return ""

    return f"{y}/{mo:02d}/{d:02d}"
```

extract_birth_date(text):

First, the input text is digit-normalized so that Persian or Arabic digits are converted into standard Latin digits.
Then, a regular expression is used to detect the date pattern.
After that, a simple validation step is applied:

- year must be between 1300 and 1499
- month must be between 1 and 12
- day must be between 1 and 31

Finally, the extracted date is standardized into the yyyy/mm/dd format.

```python
def fix_birth_text(s: str) -> str:
    s = normalize_digits(s).replace(" ", "")
    b1 = extract_birth_date(s)
    if b1:
        return b1

    # اگر نامعتبر بود، 5 را 0 فرض کن
    b2 = extract_birth_date(s.replace("5", "0"))
    if b2:
        return b2

    return ""
```

fix_birth_text(s):

- First, the function tries to extract the birth date directly from the input text.
-

If that fails, it uses a correction strategy. Sometimes OCR may incorrectly read the digit · as ۵. Therefore, the function replaces 5 with 0 and tries to extract the birth date again.

9) NATIONAL ID VALIDATION AND CORRECTION

```python
def iran_national_id_is_valid(code: str) -> bool:
    if not code or not re.fullmatch(r"\d{10}", code):
        return False
    if code == code[0] * 10:
        return False
    check = int(code[-1])
    s = sum(int(code[i]) * (10 - i) for i in range(9))
    r = s % 11
    return (r < 2 and check == r) or (r >= 2 and check == (11 - r))
```

National ID Validation

iran_national_id_is_valid(code):

The national ID number must contain exactly 10 digits.
The digits must not all be the same, such as 1111111111.
The function checks the control digit using the common Iranian national ID checksum algorithm based on mod 11.

```python
def fix_nid_by_05(code10: str) -> str:
    """
    معتبر نبود، روی رقم‌های '5' احتمال می‌دهیم '0' بوده
    حالت‌ها را تست می‌کنیم تا کد معتبر پیدا شود.
    """
    if not code10 or not re.fullmatch(r"\d{10}", code10):
        return ""
    if iran_national_id_is_valid(code10):
        return code10

    idxs = [i for i, ch in enumerate(code10) if ch == "5"]
    if not idxs:
        return ""
    if len(idxs) > 8:
        idxs = idxs[:8]

    base = list(code10)
    for mask in range(1, 1 << len(idxs)):
        temp = base[:]
        for bit in range(len(idxs)):
            if (mask >> bit) & 1:
                temp[idxs[bit]] = "0"
        cand = "".join(temp)
        if iran_national_id_is_valid(cand):
            return cand
    return ""
```

Possible Correction of 5 Instead of 0

fix_nid_by_05(code10):

If the national ID number is not valid, the function checks every position where the digit 5 appears and tests different possibilities in case that digit was actually meant to be 0.
Using a bitmask approach, it tries all possible combinations of converting some 5 digits into 0 digits until a valid national ID number is found.

```python
def extract_nid_from_digits_stream(d: str) -> tuple[str, str]:
    """
    d: فقط رقم
    خروجی: (nid_fixed, raw10)
    """
    if len(d) < 10:
        return "", ""
    for i in range(0, len(d) - 9):
        cand = d[i:i+10]
        if iran_national_id_is_valid(cand):
            return cand, cand
        fixed = fix_nid_by_05(cand)
        if fixed:
            return fixed, cand
    return "", ""
```

Finding the National ID Number Inside a Long Digit Stream

extract_nid_from_digits_stream(d):

This function receives a string that contains only digits. For example, OCR may have recognized extra unrelated digits in addition to the national ID number.
The function slides over all possible 10-digit substrings.
If a valid national ID number is found, it returns that value.
If the 10-digit substring is not valid, the function tries to repair it using fix_nid_by_05.
Output:
(nid_fixed, raw10)
This means that the function returns both the corrected version of the national ID number and the raw 10-digit version extracted from the OCR output.

```python
def append_to_excel(row: dict):
    df_new = pd.DataFrame([row])
    if os.path.exists(EXCEL_PATH):
        df_old = pd.read_excel(EXCEL_PATH)
        df_all = pd.concat([df_old, df_new], ignore_index=True)
    else:
        df_all = df_new
    df_all.to_excel(EXCEL_PATH, index=False)
```

10) SAVING DATA TO EXCEL

append_to_excel(row):

- If the Excel file already exists, the function reads it and adds the new row to the existing data.
- If the Excel file does not exist, the function creates it from scratch.
- After adding the new record, the function writes the updated data back to the Excel file.
- **Note:**

This method can become slow when the number of records is very large, because the entire file is read and written each time. However, for a lightweight academic project, this approach is usually acceptable.

```python
def crop_norm(img, x1, y1, x2, y2):
    """
    Crop (0..1) با مختصات نرمال‌شده
    """
    h, w = img.shape[:2]
    X1, Y1 = int(x1 * w), int(y1 * h)
    X2, Y2 = int(x2 * w), int(y2 * h)

    X1 = max(0, min(w - 1, X1))
    X2 = max(0, min(w, X2))
    Y1 = max(0, min(h - 1, Y1))
    Y2 = max(0, min(h, Y2))

    if X2 <= X1 or Y2 <= Y1:
        return img
    return img[Y1:Y2, X1:X2]
```

11) IMAGE PROCESSING FUNCTIONS

crop_norm

crop_norm(img, x1,y1,x2,y2):

- This function receives normalized coordinates between 0 and 1.
- It converts these normalized coordinates into actual pixel coordinates.
- It also applies safety checks to make sure that the selected region does not go outside the image boundaries.
- **Finally, it returns the selected region of interest, or ROI**

```python
def preprocess_for_ocr(img_bgr: np.ndarray) -> np.ndarray:
    """
    پیش‌پردازش ملایم برای فارسی
    """
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    gray = clahe.apply(gray)
    gray = cv2.GaussianBlur(gray, (3, 3), 0)
    return cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
```

preprocess_for_ocr

For Persian text:
The image is first converted to grayscale.
CLAHE is applied to improve contrast.
A light blur is applied to reduce noise.
Finally, the image is converted back to BGR format, because EasyOCR can process images more easily in RGB/BGR-compatible formats.

```python
def preprocess_digits_roi(roi_bgr: np.ndarray) -> np.ndarray:
    """
    پیش‌پردازش مخصوص اعداد: threshold + invert + close
    """
    gray = cv2.cvtColor(roi_bgr, cv2.COLOR_BGR2GRAY)
    gray = cv2.GaussianBlur(gray, (3, 3), 0)
    _, th = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    if th.mean() > 127:
        th = 255 - th

    k = cv2.getStructuringElement(cv2.MORPH_RECT, (2, 2))
    th = cv2.morphologyEx(th, cv2.MORPH_CLOSE, k, iterations=1)
    return cv2.cvtColor(th, cv2.COLOR_GRAY2BGR)
```

preprocess_digits_roi

For numerical fields:
The image is converted to grayscale and a blur filter is applied.
Otsu thresholding is then used to separate the digits from the background.
If the background is white, the image is inverted so that the digits and background have better contrast.
A morphological closing operation is applied to connect small gaps or broken parts inside the digits, which can improve OCR performance.

```python
def ocr_digits_easyocr(img_bgr: np.ndarray) -> str:
    img2 = cv2.resize(img_bgr, None, fx=2.5, fy=2.5, interpolation=cv2.INTER_CUBIC)
    txts = reader.readtext(
        img2,
        detail=0,
        paragraph=False,
        allowlist="0123456789٠١٢٣۴۵۶۷۸۹",
        text_threshold=0.45,
        low_text=0.30,
        link_threshold=0.40,
        mag_ratio=1.0,
    )
    return clean_text(" ".join(txts))
```

ocr_digits_easyocr

- The image is enlarged by a factor of 2.5.
- The allowlist is restricted to digits only, including both Persian and Latin digits.
- Some threshold-related parameters are adjusted to improve digit recognition accuracy.

12) DETECTING THE CARD QUADRILATERAL AND APPLYING WARP

This is one of the most important parts of the project.

Lines 286 to 340

find_card_quad

- The image is first converted to grayscale and then blurred.
- Canny edge detection is applied using adaptive thresholds based on the median value of the image, which is more flexible than using fixed threshold values.
- A morphological closing operation is applied to close gaps between detected edges.
- The function then searches for large contours in the image.
- If a four-sided contour is found, it is ordered correctly and returned as the detected card quadrilateral.
- If no suitable four-sided contour is found, the function uses a fallback approach based on minAreaRect. In this method, the best rectangle is selected using a scoring system based on:
    - similarity between the rectangle’s aspect ratio and the target card aspect ratio, where CARD_AR = WARP_W / wARP_H;
    - the size of the detected rectangle compared to the whole image.

```python
def warp_card(img_bgr: np.ndarray):
    quad = find_card_quad(img_bgr)
    if quad is None:
        return img_bgr

    dst = np.array([
        [0, 0],
        [WARP_W - 1, 0],
        [WARP_W - 1, WARP_H - 1],
        [0, WARP_H - 1]
    ], dtype="float32")

    M = cv2.getPerspectiveTransform(quad, dst)
    warped = cv2.warpPerspective(img_bgr, M, (WARP_W, WARP_H))
    return warped

# ----------------------
```

12) DETECTING THE CARD QUADRILATERAL AND APPLYING WARP

warp_card

If a quad is found:

- A perspective transformation matrix is created.
- The card is transformed into a fixed size of 1200 × 760 pixels.

Main advantage:

After the warp operation, the positions of the fields become almost fixed. Therefore, fixed ROIs can be used effectively.

```python
def ocr_boxes(img_bgr: np.ndarray):
    rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    results = reader.readtext(rgb, detail=1, paragraph=False)
    boxes = []
    for (bbox, text, conf) in results:
        text = clean_text(text)
        if not text:
            continue
        xs = [p[0] for p in bbox]
        ys = [p[1] for p in bbox]
        x1, y1, x2, y2 = int(min(xs)), int(min(ys)), int(max(xs)), int(max(ys))
        cx, cy = (x1 + x2) / 2.0, (y1 + y2) / 2.0
        boxes.append({
            "text": text, "conf": float(conf),
            "x1": x1, "y1": y1, "x2": x2, "y2": y2,
            "cx": cx, "cy": cy
        })
    return boxes
```

13) OCR BOXES AND FIELD EXTRACTION BASED ON “LABEL → VALUE”

ocr_boxes

EasyOCR is called with detail=1. For each detected text segment, the function receives:
- the bounding box,
- the recognized text,
- and the confidence score.

Then, the coordinates x1, y1, x2, y2 and the center point cx, cy are calculated.

- After that, a list of boxes is created.

⚠️ Note:

In the file, ocr_boxes is defined twice: one older version and one newer version. The second definition overrides the first one, so the newer version is the one that is actually used.

14) FUZZY MATCHING OF LABELS (NATIONAL ID / NAME / ETC.)

PERSIAN CHARACTER NORMALIZATION

FA_CHAR_MAP:

- ک → ك ,ی → ي, and similar character conversions are applied. The zero-width non-joiner (ZWNJ) is also removed.
- This helps normalize the OCR output when Arabic and Persian characters are mixed, making the recognized text more consistent.

LABEL_PATTERNS

For each field, several possible label forms are defined:
nid: “کد ملی“ ,”شماره ملی”, etc.
name: “نام”
family: “نام خانوادگی”, etc.
birth: “تاریخ تولد”, etc.

- father: “پدر“ ,”نام پدر”, etc.

MIN_LABEL_SCORE

This value defines the minimum fuzzy matching score required to accept a detected text as a valid label.
For the name field, the threshold is set very strictly, for example 92, because the word “نام” is short and may easily be confused with labels such as “نام خانوادگی” or “نام پدر”.

EXPECTED_Y و Y_TOL

This is a useful technique for reducing extraction errors:

- For example, the program expects the “name” label to usually appear around y = 0.33 of the warped card.
- Therefore, if OCR detects a word similar to “نام” but its y-position is too far from the expected location, the program rejects it.
- This significantly reduces serious label-matching errors.

14) FUZZY MATCHING OF LABELS (NATIONAL ID / NAME / ETC.)

```
FA_CHAR_MAP = str.maketrans({
    "ي": "ی", "ك": "ک", "ة": "ه", "ۀ": "ه",
    "ؤ": "و", "إ": "ا", "أ": "ا", "آ": "ا",
    "\u200c": "",  # ZWNJ
})

LABEL_PATTERNS = {
    "nid":    ["شمارهملی", "شماره ملی", "کدملی", "کد ملی", "ملی"],
    "name":   ["نام"],
    "family": ["نامخانوادگی", "نام خانوادگی", "خانوادگی", "خانواد"],
    "birth":  ["تاریختولد", "تاریخ تولد", "تولد"],
    "father": ["نامپدر", "نام پدر", "پدر"],
}

# سختگیری برای "نام" چون کوتاه است و زیاد با بقیه قاطی می‌شود
MIN_LABEL_SCORE = {"nid": 72, "name": 92, "family": 72, "birth": 72, "father": 78}

# جای تقریبی لیبل‌ها روی کارت warp (نرمال‌شده 0..1)
EXPECTED_Y = {
    "nid": 0.22,
    "name": 0.33,
    "family": 0.43,
    "birth": 0.56,
    "father": 0.67,
}
Y_TOL = {
    "nid": 0.16,
    "name": 0.10,
    "family": 0.10,
    "birth": 0.11,
    "father": 0.11,
}
```

14) FUZZY MATCHING OF LABELS (NATIONAL ID / NAME / ETC.)

LINES 455 TO 486

label_match_score

For each text box, this function performs the following steps:

First, the text is normalized by removing spaces and punctuation marks.

If most of the text consists of digits, it is not considered a label.

Then, fuzzy matching is applied between the detected text and the predefined label patterns.

Error-prevention rules are also applied:

- If kind = name and the detected text is highly similar to father or family labels, it is rejected.
- If kind = father, the detected text must be similar to “پدر” father. Simply being similar to “نام” name is not enough.

15) LINE-AWARE DETECTION AND SELECTING THE VALUE NEXT TO THE LABEL

On the ID card, the structure is usually as follows:
[Value] [Label]
In Persian layouts, the value is usually located to the left of the label.

assign_line_ids

- This function calculates the average height of the detected text boxes.
- Then, it sorts the boxes based on their vertical center coordinate, cy.
- Using a threshold, it groups boxes that are located on the same horizontal line and assigns a line_id to each group.

```python
def assign_line_ids(boxes):
    if not boxes:
        return
    hs = [max(8, b["y2"] - b["y1"]) for b in boxes]
    med_h = float(np.median(hs)) if hs else 18.0
    thr = max(16.0, 0.65 * med_h)  # آستانه‌ی تشخیص خط

    boxes_sorted = sorted(boxes, key=lambda b: b["cy"])
    line_id = 0
    current_y = None

    for b in boxes_sorted:
        if current_y is None or abs(b["cy"] - current_y) > thr:
            line_id += 1
            current_y = b["cy"]
        else:
            current_y = 0.7 * current_y + 0.3 * b["cy"]
        b["line_id"] = line_id
```

pick_value_for_label

LINE 506 TO 548

- For a given label, this function only checks the text boxes that have the same line_id as the label.
- Important condition:
  The value must be located to the left of the label:
- b["x2"] < label_box["x1"]
- Among the candidate boxes, the closest one is selected.

If multiple nearby text parts belong to the same value, they are joined together. This is useful when the extracted value consists of more than one word.

extract_fields_by_labels LINE 553 TO 590

- For each kind of field, this function finds the best matching label using a combination of fuzzy score, OCR confidence, preference for labels located on the right side, and y-position constraints.
- After the best label is found, the corresponding value is extracted using the pick_value_for_label function.
- Output:
- values: The extracted text values for fields such as nid, name, and others.
- label_boxes: The detected boxes that were selected as field labels.
- picked_parts: The text boxes that were selected as the corresponding field values.

16) ERROR HANDLING FOR THE “FATHER’S NAME” FIELD

```python
def is_suspect_father(father: str, family: str) -> bool:
    f = clean_text(father)
    if not f:
        return True
    # پدر معمولاً خیلی کوتاه‌تر از نام خانوادگی است
    if len(f) > 18 or f.count(" ") >= 3:
        return True
    # اگر خیلی شبیه نام خانوادگی شد، یعنی از خط اشتباه آمده
    if family and _sim(f, family) >= 78:
        return True
    return False
```

is_suspect_father(father, family):

- If the extracted father’s name is empty, too long, or consists of too many words, it is considered suspicious.
- If it is very similar to the family name, it is also considered suspicious. This may indicate that the value was incorrectly taken from another line.
- If the extracted father’s name is suspicious, the function sets:
- father = ""

This allows the program to read the father’s name again later using a fixed ROI.

```python
def draw_debug(img, boxes, label_boxes, picked_parts, roi_rects=None):
    dbg = img.copy()

    # OCR boxes (آبی)
    for b in boxes:
        cv2.rectangle(dbg, (b["x1"], b["y1"]), (b["x2"], b["y2"]), (255, 0, 0), 1)

    # Labels + picked values (سبز)
    for kind, lb in label_boxes.items():
        if lb is not None:
            cv2.rectangle(dbg, (lb["x1"], lb["y1"]), (lb["x2"], lb["y2"]), (0, 255, 0), 3)
            cv2.putText(dbg, kind.upper(), (lb["x1"], max(20, lb["y1"] - 8)),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        for p in picked_parts.get(kind, []):
            cv2.rectangle(dbg, (p["x1"], p["y1"]), (p["x2"], p["y2"]), (0, 255, 0), 2)

    # ROI (زرد) ها
    if roi_rects:
        for (x1, y1, x2, y2, label) in roi_rects:
            cv2.rectangle(dbg, (x1, y1), (x2, y2), (0, 255, 255), 2)
            cv2.putText(dbg, label, (x1, max(20, y1 - 8)),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)

    return dbg
```

17) CREATING THE DEBUG IMAGE

draw_debug:

- This function draws all OCR boxes in blue.
- The selected labels and their corresponding values are drawn in green.
- The fixed ROIs are drawn in yellow.
- Finally, the debug image is saved as last_debug.png.

18) FIXED ROIS FOR STABLE READING

Even if label-based extraction fails, fixed ROIs can help recover important fields.

```python
def extract_best_nid_from_roi(warped_bgr):
    """
    ROI عدد کد ملی (ثابت روی کارت warp شده)
    """
    roi = crop_norm(warped_bgr, 0.45, 0.16, 0.86, 0.30)
    roi_pp = preprocess_digits_roi(roi)
    raw_txt = ocr_digits_easyocr(roi_pp)
    d = digits_only(raw_txt)
    fixed, raw10 = extract_nid_from_digits_stream(d)
    return fixed, raw10
```

- **extract_best_nid_from_roi(warped):**
- The ROI for the national ID number is defined as:
- x: 0.45 to 0.86
- y: 0.16 to 0.30
- This region is preprocessed specifically for numerical recognition.
- Then, digits-only OCR is applied.
- The function searches through the digit stream and extracts a valid national ID number. If the detected number is not valid, it attempts to correct it and then returns the repaired version.

```python
def extract_best_birth_from_roi(warped_bgr):
    """
    ROI (ثابت) تاریخ تولد
    """
    roi = crop_norm(warped_bgr, 0.45, 0.44, 0.86, 0.60)
    roi_pp = preprocess_digits_roi(roi)
    roi2 = cv2.resize(roi_pp, None, fx=2.0, fy=2.0, interpolation=cv2.INTER_CUBIC)
    txt = reader.readtext(
        roi2,
        detail=0,
        paragraph=False,
        allowlist="0123456789٠١٢٣۴۵۶٧٨٩/-. ",
    )
    return fix_birth_text(" ".join(txt))
```

extract_best_birth_from_roi(warped):

- The ROI for the birth date is defined as:
- x: 0.45 to 0.86
- y: 0.44 to 0.60
- An allowlist is used that includes only digits and date separators such as /, -, and ..
- After OCR is applied to this region, the extracted text is passed to fix_birth_text in order to normalize and correct the birth date.

18) FIXED ROIS FOR STABLE READING

ocr_fa_roi(warped, ...):

- This function enlarges the selected ROI.
- Then, paragraph-based OCR is applied, which works better for Persian text.
- This function is used as a fallback method for extracting the name, family name, and father's name fields.

```python
def ocr_fa_roi(warped_bgr, x1, y1, x2, y2) -> str:
    roi = crop_norm(warped_bgr, x1, y1, x2, y2)
    roi = cv2.resize(roi, None, fx=2.0, fy=2.0, interpolation=cv2.INTER_CUBIC)
    txts = reader.readtext(roi, detail=0, paragraph=True)
    return clean_text(" ".join(txts))
```

19) MAIN ENDPOINT: /PROCESS-PHOTO

This endpoint receives the uploaded file using several possible field names:

photo or file or image or imageFile

This is done to make the backend compatible with different upload forms and with camera.html.

COMPLETE FLOW:

1. Read the uploaded image and save it as last_received.png.
2. Apply warp_card to straighten and normalize the card.
3. Apply preprocess_for_ocr to improve OCR quality.#
4. Use ocr_boxes and extract_fields_by_labels to perform the initial field extraction based on detected labels.
5. If the extracted father’s name is suspicious, it is cleared.
6. Extract and correct the birth date from the label-based result:
birth_label = fix_birth_text(values["birth"])
7. The national ID number is also extracted from the label-based OCR text as a backup.
8. The main national ID number is read from the fixed ROI.
9. If the birth date is not successfully extracted from the label-based result, it is read from the fixed ROI.
10. Fallback extraction is applied for name fields:
 - If name does not look like a valid Persian name, the name ROI is used.
 - If family or father is empty or too short, the corresponding fixed ROIs are used.
11. A debug image is generated and saved.
12. A record is created containing:
 - timestamp
 - national_id final validated value
 - raw_nid_digits raw OCR digits
 - name, family, father, and birth_date
 - raw_text all OCR-recognized text
13. The record is appended to the Excel file.

20) RUNNING THE SERVER WITH HTTPS

1. In this configuration, the FastAPI server runs on port 8443 with TLS/HTTPS enabled.
2. For this setup to work, the certificate file cert.pem and the private key file key.pem must be located in the same directory as the server file. These files are used by the server to establish secure HTTPS connections.

* For this setup to work, the certificate file cert.pem and the private key file key.pem must be located in the same directory as the server file. These files are used by the server to establish secure HTTPS connections.

```
# ==========================
if __name__ == "__main__":
    uvicorn.run(
        app,
        host="0.0.0.0",
        port=8443,
        ssl_keyfile=os.path.join(BASE_DIR, "key.pem"),
        ssl_certfile=os.path.join(BASE_DIR, "cert.pem"),
    )

```

GUIDE FOR RUNNING THE PROGRAM:

1. Connect both the mobile phone and the laptop to the same Wi-Fi network.
2. Make sure that Python and Git are installed on the laptop.
3. Open a terminal in the project directory and generate the HTTPS certificate files using OpenSSL.

For Git Bash or Command Prompt:

```
"C:\Program Files\Git\usr\bin\openssl.exe" req -x509 -newkey rsa:2048 -nodes -out cert.pem -keyout key.pem -days 365 -subj "/CN=localhost"
```

For PowerShell:

```
& "C:\Program Files\Git\usr\bin\openssl.exe" req -x509 -newkey rsa:2048 -nodes -out cert.pem -keyout key.pem -days 365 -subj "/CN=localhost"
```

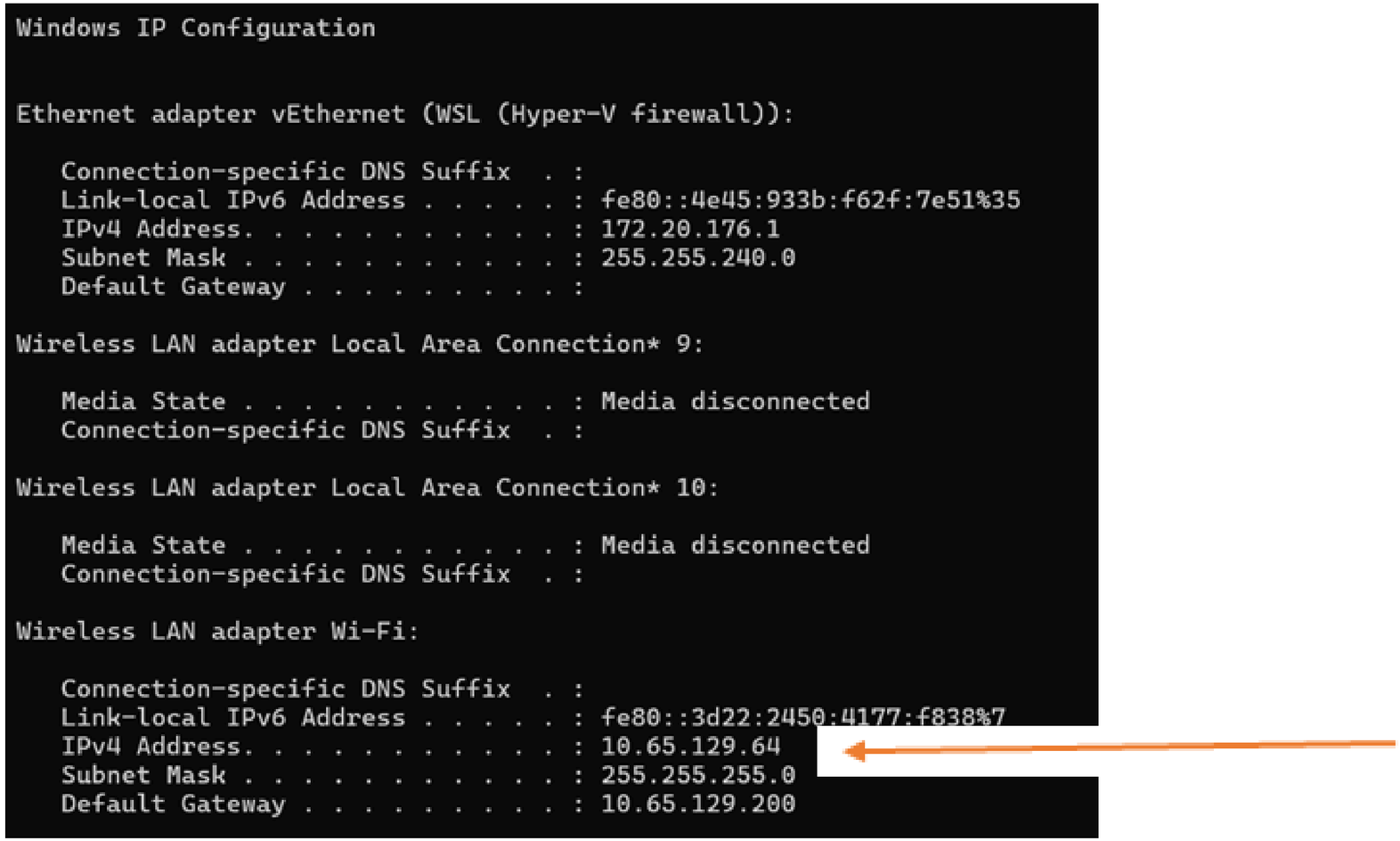
4. Start the backend server by running:

```
python server.py
```

5. In the laptop terminal, run the following command to find the IPv4 address of the laptop:

GUIDE FOR RUNNING THE PROGRAM:

6. Copy the computer’s IPv4 address:



GUIDE FOR RUNNING THE PROGRAM:

7. Open the camera.html file and replace the IP address in the relevant line with your laptop's IPv4 address.

```
const SERVER_URL = 'https://192.168.125.20:8443/process-photo';
```

8. On the mobile phone, open Firefox. Chrome may not work in this case because of its security restrictions for local HTTPS/camera access. In the address bar, enter:

https://10.65.129.64:8443/camera.html

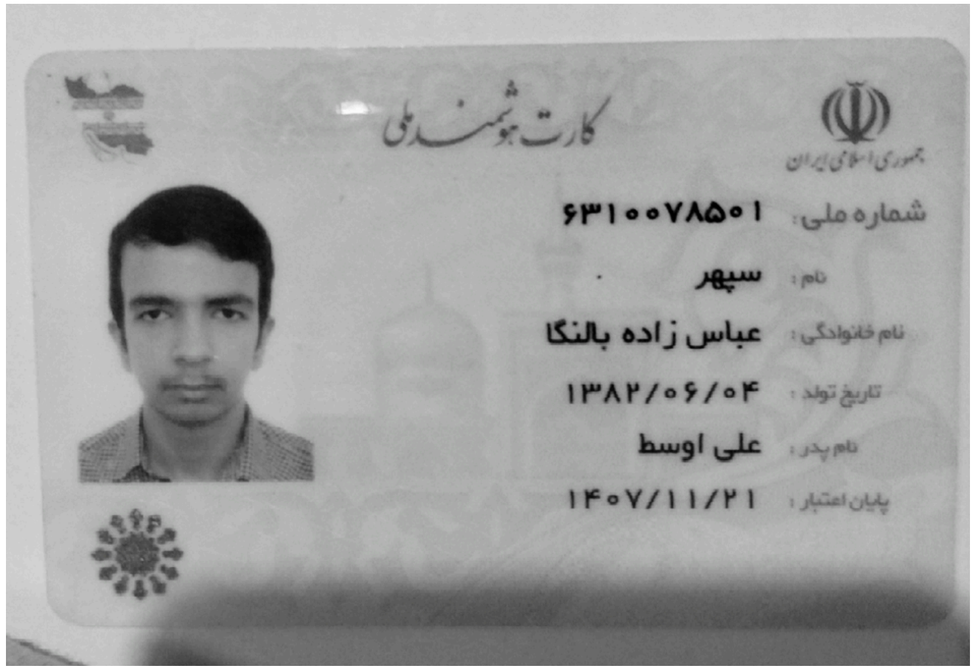
Replace 10.65.129.64 with the IPv4 address of your own laptop.

9. Since the HTTPS certificate is self-signed, the browser will display a security warning. Click Advanced, then select Accept the Risk and Continue.

10. Click the capture button once. The browser will request camera permission. Allow access to the camera.

After these steps, the program will be ready to use.

INPUT



EXCEL *OUTPUT FILE*

	A	B	C	D	E	F	G
1	timestamp	national_id	raw_nid_digits	name	family	father_name	birth_date
2	2026-01-01 21:34:1	6310078496	6315578496	سینا	عباس زاده بالنگا	اوسط علی	1382/06/04
3	2026-01-01 21:35:3	6310078501	6315578501	سپهر	عباس زاده بالنگا	اوسط علی	1382/06/04

Installing Dependencies

The dependencies used in this code are:

- **cv2 (OpenCV)**
- **numpy**
- **pandas**
- **fastapi**
- **easyocr**
- **uvicorn**
- **rapidfuzz optional**

To install the main dependencies, you can use the following command:

pip install opencv-python-headless numpy pandas fastapi easyocr uvicorn

Installing rapidfuzz optional:

If you want to use the fuzzy matching feature, which is used in the code for matching field labels, you can install rapidfuzz with the following command:

pip install rapidfuzz

3. Installing OpenCV

If you want to use the full version of OpenCV with GUI support, you can install opencv-python.

Otherwise, if you do not need GUI features, you can use opencv-python-headless:

pip install opencv-python-headless

4. Installing Additional Dependencies

If additional tools are needed, for example GPU support for EasyOCR, the following command can be used:

pip install torch torchvision torchaudio

5. HTTPS Configuration

To run the server with HTTPS, SSL certificate files are required. The two files key.pem and cert.pem, which are used for HTTPS, must be placed in the same directory where the server script is executed.

If you do not have your own SSL certificates, you can generate local test certificates for development purposes.

When HTTPS is enabled, the server runs on port 8443, and the API can be accessed through the following address:

https://localhost:8443